Ders 3 Genişletilmiş Notu: Bilgi Çıkarımı ve İstatistiksel Dil Modelleri

Ders 3 Genişletilmiş Notu: Bilgi Çıkarımı ve İstatistiksel Dil Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Dersten Bağlantı: Hayat Bilgisi ve Monotonik Olmayan Çıkarım
- 2. Bilgi Çıkarımı Nedir?
- 3. Bilgi Çıkarımı Yöntemleri
- 4. Şablonların Üretkenliği ve Güvenilirliği
- 5. Web Sayfalarından Bilgi Çıkarımı ve Üretici Modeller Tartışması
- 6. Şablonların Otomatik Bulunması ve Sonuçların Süzülmesi
- 7. İstatistiksel Dil Modelleri
- 8. Zincir Kuralı ve Frekansla Olasılık Hesabı
- 9. Markov Varsayımı
- 10. Seyreklik: Ekonomi Haberleri Örneği
- 11. Bigram Modeliyle Cümle Olasılığı
- 12. Sıfırlarla Baş Etmek: Yumuşatma
- 13. En Olası Dizilişi Bulmak
- Örnek Sorular
- Dersin Toparlaması

1. Geçen Dersten Bağlantı: Hayat Bilgisi ve Monotonik Olmayan Çıkarım

Geçen derste hayat bilgisi (commonsense) veritabanlarından söz edilmişti. Bu veritabanları en azından “şu kavramla şu kavram arasında şu türden bir ilişki vardır” biçimindeki bilgileri tutar. Bu bilgileri toplamak için çeşitli yaklaşımlar vardı; bunların bir kısmı bilgiyi gönüllülerden almaya dayanıyordu ve oyunlaştırma da bu yaklaşımlardan biriydi. CSoyun’da kullanıcılar kavramlar arasında ilişki ve yeni kavram girebiliyordu.

Hoca, CSoyun’daki “Tahminlerim” oyununu ve monotonik / monotonik olmayan çıkarım ayrımını hatırlattı (ekrandan, 00:10:00). Monotonik çıkarımda öncüller doğruysa sonuç kesinlikle doğrudur. Bu yüzden veritabanına yeni bilgi eklemek açısından monotonik çıkarımın çok fazla katkısı yoktur; belki kontrol için kullanılabilir. Asıl katkı monotonik olmayan çıkarımdan gelir: sistem tahminler, yani olası ilişkiler üretir ve bunların doğru olup olmadığını kullanıcılara sorar. Slayttaki tabloda bu tahminlerin dört üretilme biçimi vardı; örneğin (martı - altkavramıdır - kuş) ve (martı - bu ne yapabilir - uçmak) bilgilerinden (kuş - bu ne yapabilir - uçmak) tahmini üretilir. Geçen ders hayat bilgisi veritabanlarının kalitesinin nasıl ölçüldüğüne kadar gelinmişti; bu ders bilgi çıkarımıyla devam etti.

2. Bilgi Çıkarımı Nedir?

Bilgi çıkarımı (information extraction), hayat bilgisi veritabanlarını oluşturmak için de, başka amaçlar için de kullanılabilecek bir yaklaşımdır. Hoca bunu en temelde şöyle tanımladı: yapısal

olmayan bir kaynağı yapısal hâle çevirmeye çalışmak.

Yapısal kaynaktan kastedilen tablo biçimindeki yapılardır: bir SQL veritabanı ya da bir Excel tablosu. Böyle bir yapıda her hücrenin ne olduğu ve aynı satırdaki diğer hücrelerle nasıl bir ilişkisi olduğu bellidir; bunu biz tanımlarız ve programlarımızı ona göre yazarak tablonun tutarlı biçimde doldurulmasını sağlarız. Yapısal olmayan kaynak ise düz metindir. Metindeki bilgilerin birbirleriyle nasıl ilişkili olduğu verilmemiştir; bilgi çıkarımı tam olarak bunu bulmaya çalışır.

Slayttaki örnek (ekrandan, 00:11:50):

> 21 yaşındaki inşaat işçisi Kazım Yaprak, evine dönerken para meselesi yüzünden tartıştığı arkadaşı Hilmi Baker tarafından bıçaklanarak öldürüldü.

Bu paragraftan şu tablo oluşturulmak istenir:

Alan	Değer
Katil	Hilmi Baker
Kurban	Kazım Yaprak
Sebep	Para
Suç Aleti	Bıçak

Soru: Böyle bir şeyi yapabilmek ne işe yarar? **Cevap:** Öğrenciler, verinin üzerinde çok daha kolay işlem yapılabileceğini, örneğin suç aletlerinin ya da katillerin tutulduğu bir veritabanı kurulabileceğini söyledi. Hoca ekledi: veri çok daha işlenebilir hâle gelir ve istatistik çıkarmak çok kolaylaşır. Aslında metinleri okumadan onların bir tür özetini çıkarmış olursunuz. Veriden ne çıkarmak istediğinizi tarif edip bunu yeterince doğru biçimde alabilirseniz, tek bir paragraf yerine bir milyon paragraftan bu tür tablolar çıkararak çok hızlı ve sağlam istatistik bilgisi elde edebilirsiniz.

3. Bilgi Çıkarımı Yöntemleri

Bu tür bilgileri çıkarmak için birçok yöntem vardır ve bazıları birbirine oldukça benzer.

3.1 Düzenli ifadeler (regex)

İlk yöntem düzenli ifadelerdir (regular expressions). Verinin içindeki düzenliliği ve bu düzenliliğin varyasyonlarını kesin olarak biliyorsanız regex ile o bilgiye erişebilirsiniz. Telefon numarasının regex'ini yazmak kolaydır; posta kodu ve e-posta adresi için de durum fena değildir. Ama bir adresin, bütün adresleri kapsayacak tek bir regex'ini yazmak hiç kolay değildir, çünkü adreslerin varyasyonu çok daha yüksektir. Varyasyonların bilindiği ve yeterince kapsanabildiği durumlarda ("içinde şu kadar sayı varsa, bu sayılar şundan büyük değilse, sonra üç karakter ve bir nokta geliyorsa" gibi) regex kullanmak oldukça mantıklıdır.

3.2 Yazım dilindeki şablonlar

İkinci yöntem, rakam ya da büyük-küçük harf gibi özelliklere değil, dildeki kelimelere dayanan şablonlardır. Slayttaki fikir şudur: istenen ilişki türü için şablonlar bulunur ve büyük metinlerde bu şablonlara uyan çiftler çıkarılır (ekrandan, 00:16:20).

Örnek şablon "ve diğer"dir. Bir arama motorunda "ve diğer" aratıldığında, bu ifadenin geçtiği sayfa başlıkları ve içerikleri gelir. Şablonun solundaki ve sağındaki ifadeler her zaman değil ama genelde bir IsA ("bu, şu tür bir şeydir") ilişkisi içindedir. Slayttaki arama sonuçlarından:

- "Gıda ve diğer mühendislikler" → gıda bir tür mühendisliktir.
- "Ses kayıtları ve diğer multimedya içeriği" → ses kayıtları bir tür multimedya ürünüdür.
- "Spor ve diğer aktiviteler" → spor bir tür aktivitedir.
- "Dergiler ve diğer periyodikler" → dergiler bir tür periyodiktir.
- "Güzel sanatlar ve diğer konular" → güzel sanatlar bir tür konudur. Hoca bunun çok anlamlı olmadığını söyledi.
- "Sözleşmeler ve diğer hukuki belgeler" → sözleşmeler bir tür hukuki belgedir.

Bu kadar basit bir şablonla bile bu ilişkiler bulunabilir. Ama şablon her zaman doğru sonuç vermez. Hoca, bu şablonla elde edilen ikililerin belki %80'inin doğru, bir kısmının yanlış olabileceğini söyledi. Yazım dilindeki şablonlar, ilişki türüne ve yazılan şablonun kalitesine göre farklı kalitede ve farklı kapsayıcılıkta içerik verir.

3.3 Öğelerine ayrılmış metinler

Üçüncü yöntem, öğelerine ayrılmış metinleri kullanmaktır. Cümleleri özne, nesne, yüklem gibi öğelerine ayırabilen bir aracınız olduğunu düşünün (hoca böyle araçların bulunduğunu söyledi). O zaman şunu diyebilirsiniz: yüklemi "yemek" olan cümlelerdeki belirtili nesneler yiyecektir. Slayttaki ifadeyle: "ye" fiilinin nesneleri yiyecek olarak sınıflandırılabilir (ekrandan, 00:19:50).

Bu fikrin daha basit bir biçimi şudur: bir arama motorunda ya da bir derlemde "yerken" kelimesinden önceki kelimeleri yiyecek kabul etmek. Slayttaki arama sonuçlarında "döner yerken", "kraker yerken", "zeytin yerken", "mantar yerken" gibi doğru sonuçlar çıkar. Ama "Yerken" soyadına ait bir soyağacı arama sayfası da gelir ve oradan yiyecek olmayan bir şey çıkar. Hoca bu yöntemin belli yerlerde hata yapacağını ama fena sonuç üretmediğini söyledi. "içerken" aratıldığında da "çay içerken", "kahve içerken" gibi sonuçların yanında "sigara içerken" gibi pek uygun olmayanlar gelir. Yani belli fiillerden önceki kelimelerin belli türden yapılara sahip olduğu söylenebilir.

4. Şablonların Üretkenliği ve Güvenilirliği

Regex olsun, öğelere ayırma olsun, bu yöntemlerin hepsinde karşımıza şablonlar çıkar. Şablonların iki özelliği vardır:

- **Üretkenlik:** Şablonu kullanarak kaç tane bilgi (ikili, üçlü vb.) elde edildiği.
- **Güvenilirlik:** Şablonun ürettiği bilginin doğruluğu.

Bir tarafta miktar, bir tarafta doğruluk vardır ve bunlar birbiriyle çelişir.

Slayttaki örnek "X'in yeri Y'dir" ilişkisi için bulunan şablonlardır (ekrandan, 00:22:10): "y deki x", "y de bulunan x", "y de x", "x y de", "x y ili sınırları içerisindedir", "y ili sınırlarında kalan x", "y ili sınırları içinde bulunan x", "y ilçesi sınırları içinde bulunan x", "x y nin sınırları içerisindedir", "x/y", "x / y", "x-y", "x y ye zz km", "x, y ye zz km", "x (y ye zz km", "x, y", "x - y", "x bulunduğu yer:y", "y-x", "x(y", "x(y)".

Hoca "x y nin sınırları içerisindedir" ile "x/y" şablonlarını karşılaştırdı. "x/y" şablonunu kullanmak, eğik çizginin geçtiği her yerde sağındaki ve solundaki kelimeleri almak demektir. Diğerinde ise "nin sınırları içerisindedir" aratılır ve önündeki iki kelime alınır. Önce hangisinin daha üretken olduğu soruldu; sonra doğruluk tartışıldı (bkz. Örnek Sorular). Sonuç: "x/y" çok daha fazla sonuç döndürür, çünkü eğik çizgi binlerce farklı durumda kullanılır; yani üretkenliği çok yüksektir. "sınırları içerisindedir" çok daha az sonuç döndürür ama döndürdüklerinin doğru olma olasılığı çok daha yüksektir.

> **Hocanın vurgusu:** Üretkenlik ve güvenilirlik birbiriyle çelişir. İkisini birden iyileştiren şablonlar bulmak çok kolay değildir. Gelen sonuçları ayrıca doğrulamak gibi teknikler kullanılabilir.

5. Web Sayfalarından Bilgi Çıkarımı ve Üretici Modeller Tartışması

5.1 Dinamik web sayfalarından bilgi çıkarımı

Ürün karşılaştırma motorları bir ürünün hangi sitede kaça satıldığını listeler. Bunu yapabilmek için, siz bir şey sorduğunuzda arka tarafta bunu örneğin on farklı siteye sorarlar ve dönen sayfaların HTML kodundan sizin sorduğunuz bilgiyi ayıklarlar. Önbellek (cache) kullanabilirler; örneğin son bir saat içinde aynı sorguyu yaptılarsa tekrar yapmazlar. Ama özünde yapılan iş, bir HTML sayfasından bilgi çıkarmaktır.

Slayttaki örnek, Amazon'un bir zamanlarki kitap sayfasının HTML kodudur (ekrandan, 00:26:20). Koddan şu bilgiler elde edilir:

```
Title:      The Age of Spiritual Machines : When Computers Exceed Human Intelligence
Author:     Ray Kurzweil
List-Price: $14.95
Price:      $11.96
```

HTML kodunun ilgili kısmı (ekrandan, kısaltılmış):

```
<b class="sans">The Age of Spiritual Machines : When Computers Exceed Human Intelligence</b><b
by <a href="/exec/obidos/search-handle-url/index=books&field-author=Kurzweil%2C%20Ray/002-6235
Ray Kurzweil</a><br>
...
<b>List Price:</b> <span class=listprice>$14.95</span><br>
<b>Our Price: <font color=#990000>$11.96</font></b><br>
<b>You Save:</b> <font color=#990000><b>$2.99 </b> (20%)</font><br>
```

Buradan şablonlar kurulur: örneğin “field-author” ile başlayan bağlantıdan sonra ilk `</a>`’ya kadar olan kısım yazardır; `<span class=listprice>` ile başlayıp sonraki `</span>`’a kadar olan kısım liste fiyatıdır. Hoca bunu şöyle özetledi: başka birinin yapısal olarak tuttuğu ama ekranda yapısal olarak vermediği veriyi tekrar yapısal hâle getirmeye çalışırsınız.

Bu, ticari anlamda bir yarıştır. İçerik üreten firmalar veritabanlarının başkaları tarafından sorgulanmasını bir açıdan istemez, ama iyi trafik getiriyorsa isterler; o zaman API’ler ya da karşılıklı anlaşmalarla erişim açabilirler. Alışveriş ve haber toplama robotları temelde bu işi yapar ve her web sayfasının yapısına özel bir çıkarım mekanizması kurmak gerekir. Bunu istemeyen siteler sayfalarında belli rastgele değişiklikler yaparak çıkarım kodunun sürekli güncellenmesini gerektiren yapılar kurar; karşı tarafta da daha genel çıkarıcılar yazılmaya çalışılır.

5.2 Üretici modeller ve gönüllü içerik tartışması

Hoca buradan üretici (generative) modellere geçti. Bu modeller, metin ya da görsel olsun, internette halka açık olan içerikleri toplayıp onlardan yeni şeyler üretir. Bu içerikler halka açıktır, ama insanlar onları bu teknolojiler yokken, buna göre yazmışlardı. Hocanın sorusu şuydu: Stack Overflow’un bütün içeriğini özümseyen, bir soru sorulduğunda cevabı oradan bulup getiren ve siteye hiçbir referans vermeyen bir sistem düşünün. İnsanlar hâlâ o siteye girip soruları cevaplamaya devam eder mi? Soru sorma o siteden zaten çok düşer, çünkü cevap doğrudan alınabilir. Aynı soru GitHub için de sorulabilir: bütün GitHub kodlarını inceleyip kullanıcının istediği kodu üreten bir sistem varsa insanlar kodlarını GitHub’a koymaya devam eder mi?

Soru (bir öğrenci): Veri zaten halka açık değil mi? Elle de toplanabilirdi. **Cevap:** Hoca bu konuda hemfikir olduğunu, halka açık verinin korunması gibi bir şeyden söz etmediğini söyledi. Haber ve e-ticaret siteleri bu işten para kazanır; onun kastettiği, para kazanma amacı olmadan, “amatör ruhla” forumlara, kendi sitelerine ya da videolara bilgi koyan gönüllülerdir.

Öğrencilerden gelen görüşler şöyle özetlenebilir:

- İçerik üretmek zordur; üretilen içerik kullanılıp üreten kişi referans olarak gösterilmezse paylaşım zamanla azalır. İnsanlar paylaşırken görünürlük ve kariyer gelişimi de bekler.
- Bir iş için harcanan zaman bir maliyettir; başkaları bu içerikten doğrudan ya da dolaylı (API üzerinden) kâr sağlıyorsa, içerik üreten kişinin bunu yapmaya isteği azalır.
- Öte yandan Reddit gibi yerlerde insanlar anonim olarak, özgeçmiş kaygısı olmadan yardım ediyor; bu yüzden etkinin sınırlı kalacağını düşünen de vardı.
- Bir öğrenci, bu araçların cevapla birlikte birkaç referans da verebileceğini önerdi.

Hoca, Google’ın web sayfalarını indekslemesini karşılaştırma için kullandı: siteler Google tarafından indekslenmeyi çok ister, çünkü karşılığında kullanıcılar onların sayfalarına gelir. Google kaynağı, kullanıcıyı o sayfaya göndermek için gösterir. Bu yeni sistemlerde ise amaç kullanıcıyı başka yere göndermemektir: “cevabın burada”. Hoca bu arayüzlerin genel kullanımının çok paralı olacağını düşünmediğini, giderek uygulamaların içine gömüleceğini tahmin ettiğini söyledi.

Referans gösterme önerisine hoca şöyle cevap verdi: bunu yapabilseler zaten yaparlar; sorun tam da burada. Bu, dersin doğrudan kapsamında olmayan ama çok kritik bir alan olan açıklanabilir yapay zekâya (explainable AI) bağlanır: “Bu cevabı nasıl, hangi kaynağı kullanarak

ürettin?” Cevap tek bir kaynaktan değil, birçok kaynağın öğrenilmesinden üretilir; hangi kaynağın ne katkı yaptığını listelemek çok zordur. Bir öğrenci üretilen cevabın metin benzerliğine bakılarak kaynak bulunabileceğini önerdi; hoca bunun olabileceğini, ama cevap kaynağın birebir aynısı olmadığı için kullanıcının sayfada aynı metni bulamayacağını belirtti. Aynı sorun sanat ürünleri gibi her tür içerik için de geçerlidir.

6. Şablonların Otomatik Bulunması ve Sonuçların Süzülmesi

Aradan sonra hoca, kendi grubunun vaktiyle yaptığı bir çalışmayı anlattı.

Fikir, şablonları elle yazmak yerine otomatik bulmaktır. Örneğin “tüm X’ler Y’dir” ilişkisi için elde doğru olduğu bilinen 15-20 X-Y ikilisi vardır. Bu ikililer aratılır; iki kavram arasındaki ifadeler şablon adaylarıdır, bunların kaç kez geçtiği de frekanslarıdır. En yüksek frekanslı ifadeler o ilişkinin şablonları olarak alınır. “Tüm X’ler Y’dir” için bulunan şablonlar arasında “ve diğer”, “ler ve diğer”, “ve benzeri”, “veya diğer” gibi ifadeler vardır; “X’in yeri Y’dir” için bulunanlar Bölüm 4’te verildi.

Sonra bulunan şablonlar Google’da aratılır ve sonuç sayfalarında şablonların sağındaki ve solundaki kelimeler alınıp bir dosyaya kaydedilir (ekrandan, 01:02:40). Hoca, IsA türünden bulunan ikililerden örnekler okudu: “bütün kontrolörler personeldir”, “bütün kemerler teçhizattır”, “bütün proteinler gıdadır”, “Azerbaycan bir bölgedir”, “komünizm bir ideolojidir” gibi. Bazı ikililer ise ters yönde çıkmıştır (örneğin “Kocaeli Spor” ve “fotoğrafçı” örneklerinde).

Bulunan ikililerin bazıları doğru, bazıları yanlıştır. Bunu makine öğrenmesiyle ayırt etmek için:

1. Önce insanlar bulunan ikililere bakar ve hangilerinin uygun olduğunu elle etiketler (human in the loop).
2. Sonra ikililerin morfolojik yapısından (ek türleri, şablon türleri gibi) özellikler çıkarılarak bir veri kümesi oluşturulur.
3. Standart makine öğrenmesi teknikleriyle sınıflandırma yapılır.

Slayttaki “Farklı sınıflandırıcılar için sonuçlar” grafiği ortalama sınıflandırma hatalarını gösterir (ekrandan, 01:03:45; değerler grafikten okunmuştur):

Sınıflandırıcı	Ortalama hata (yaklaşık)
AdaBoost	0,28
Backpropagation	0,28
CART	0,13
Nearest Neighbor	0,21
SVM	0,21

Hoca bunu yaklaşık %15 hata, %85 başarı diye özetledi ve sonuçların fena olmadığını söyledi. Böylece şablonlardan gelen sonuçların doğru ya da yanlış olduğunu bulan bir mekanizma kurulabilir ve onun üzerinden filtreleme yapılabilir.

Günümüze gelince, büyük dil modellerinin (large language models) hayat bilgisini içerip içermediği ve içermiyorlarsa bir veritabanının bu modellere nasıl katılacağı üzerine birçok araştırma sürmektedir. Slayttaki yeni girişimlerden biri 2018’de Allen Institute for Artificial Intelligence (AI2) tarafından başlatılan Alexandria’dır (ekrandan, 01:04:50). Hoca, görece yeni yaklaşımlarla çeşitli veri kümelerinde başarılar elde edildiğini ve bazılarında akıl yürütme (reasoning) mekanizmalarının da bulunduğunu söyledi. Hayat bilgisi bölümü burada tamamlandı.

7. İstatistiksel Dil Modelleri

Yeni bölüm “Anlamsal Uzaylar”dır ve ilk konusu istatistiksel dil modelleridir. Hoca, burada sinirsel dil modellerinden değil, görece eski yaklaşımlar olan istatistiksel dil modellerinden söz ettiğini vurguladı: temelleri neye dayanır, nasıl çalışır, ne yapabilir ve ne yapamaz.

İstatistiksel dil modellerinin temelde ihtiyaç duyduğu şey büyükçe bir derlemdir. Derlem varsa, modele verilen bir cümlenin var olma olasılığı bulunabilir. Slayttaki kullanım alanları (ekrandan, 01:10:00):

- **Makine çevirisi:** Bir metin Türkçeye çevrilirken bir kelimenin birden çok karşılığı olabilir ve aynı cümleden farklı çeviriler üretilebilir. Olasılığı daha yüksek olanın doğru olma ihtimali daha yüksektir varsayımıyla ilerlenir. Örnek: $P(\text{taze balık aldım}) > P(\text{yeni balık aldım})$.
- **Yazım düzeltme:** Slayttaki "Ali so…an yedi" cümlesinde ikinci kelime hatalı yazılmıştır. Örneğin Levenshtein (düzenleme) uzaklığıyla ona yakın kelimeler bulunur: "soğan", "sokan" gibi. İkisi de Türkçede geçerli kelimelerdir. $P(\text{ali soğan yedi}) > P(\text{ali sokan yedi})$ olduğu dil modeliyle bulunur ve kullanıcıya "soğan" önerilir.
- **Konuşmadan metne çevirme:** Ses sinyali benzer birden çok metne dönüşebilir; "soğan" ile "sol an" sesçe yakındır. Dil modeli $P(\text{soğan yedim}) > P(\text{sol an yedim})$ olduğunu söyler.

Hoca bunu PowerPoint'in yazım denetimiyle gösterdi. Slayttaki hatalı kelimenin önerileri "sona, soban, sodan, sofan, sokan" idi; "soğan" önerilmiyordu ve öneriler olasılığa göre sıralanmamıştı. Bunun nedeni arkada bir dil modelinin değil, kelime tabanlı bir denetimin olmasıdır. Hocanın ikinci örneği: "Maliye bakanı" doğru, "Maliye akanı" yanlıştır; ama "akanı" kelimesi tek başına Türkçede doğru olabileceği için altı çizilmez. Kelime düzeyinde doğru olan bir şey cümledeki kullanım açısından yanlış olabilir. Bunu yakalamak için arkada bir dil modeli gerekir ve bu modeller oldukça büyük oldukları için her araçta bulunmaz.

> **Hocanın vurgusu:** Kelimenin tek başına doğru olması ile bir cümlenin içinde doğru olması birbirinden farklı şeylerdir.

8. Zincir Kuralı ve Frekansla Olasılık Hesabı

8.1 İki amaç

İstatistiksel dil modellerinin iki amacı vardır (ekrandan, 01:16:10):

- **Amaç 1:** Bir cümlenin olasılığını bulmak. W cümle, $w_1, w_2, \dots, w_n$ sıralı token'ları olmak üzere $P(W) = P(w_1, w_2, \dots, w_n)$.
- **Amaç 2:** Bir kelime dizisinden sonra gelecek kelimenin olasılığını bulmak. Örneğin ilk dört kelimeden sonra beşinci kelimenin gelme olasılığı $P(w_5 \mid w_1, w_2, w_3, w_4)$.

8.2 Bayes ve zincir kuralı

Bunlar klasik Bayes kuralıyla hesaplanır:

$$P(A \mid B) = \frac{P(A,B)}{P(B)} \;\Rightarrow\; P(A,B) = P(B)\,P(A \mid B) = P(A)\,P(B \mid A)$$

Bu, birden fazla değişkene genişletilince zincir kuralı elde edilir:

$$P(A,B,C,D) = P(A)\,P(B \mid A)\,P(C \mid A,B)\,P(D \mid A,B,C)$$

Yani ABCD dizisinin olasılığı; A'nın olasılığı, A'dan sonra B'nin gelmesi, AB'den sonra C'nin gelmesi ve ABC'den sonra D'nin gelmesi olasılıklarının çarpımıdır. Genel hâli (ekrandan, 01:20:00):

$$P(w_1, w_2, \dots, w_n) = P(w_1)\,P(w_2 \mid w_1)\,P(w_3 \mid w_1, w_2) \cdots P(w_n \mid w_1, \dots, w_{n-1})$$

Örnek:

$$P(\text{ali okuldan buraya geldi}) = P(\text{ali})\,P(\text{okuldan} \mid \text{ali})\,P(\text{buraya} \mid \text{ali okuldan})\,P(\text{geldi} \mid \text{ali okuldan buray}$$

8.3 Koşullu olasılığı frekansla bulmak

$P(\text{geldi} \mid \text{ali okuldan buraya})$ nasıl bulunur? Bir olasılık, derlemdeki frekansları oranlamaktır:

$$P(\text{geldi} \mid \text{ali okuldan buraya}) = \frac{\text{frekans(ali okuldan buraya geldi)}}{\text{frekans(ali okuldan buraya)}}$$

Bu değer 0 ile 1 arasındadır (bkz. Örnek Sorular). Bağlamdan sonra "geldi" hiç gelmemişse 0, bağlamdan sonra her seferinde "geldi" geliyorsa 1 olur. "ali okuldan buraya"dan sonra gelen farklı kelimelerin her biri farklı sayıda geçer; bunlar toplama bölündüğünde toplamı 1 olan bir dağılım elde edilir. Bu dağılım, o bağlamdan sonra gelecek kelimenin bir temsilidir.

Ama burada bir sorun vardır. Slayttaki ifadeyle: bu yolla güvenilir değerler elde etmek için yeterli büyüklükte bir derlemimiz olmaz. Çoğu frekans 0 olacaktır, çünkü dil ayrık (diskrit) uzayda çok seyrektir. "Ali okuldan buraya geldi" belki derlemde geçebilir, ama on beş kelimelik bir cümlenin birebir aynı hâliyle geçme olasılığı çok düşüktür. Her kelimeden sonra her kelime gelmez; bu yüzden çok sayıda sıfırla karşılaşılır ve olasılıklar iyi hesaplanamaz.

9. Markov Varsayımı

9.1 Tanım

Slayttaki ifadeyle "Markov imdada yetişir" (ekrandan, 01:21:20). Markov varsayımı: her olasılık kendinden önceki k bileşene bağlıdır. Standart yapıda bir öncekine bağlıdır, ama k bir parametre olarak alınabilir.

$k = 0$ için her olasılık bağımsızdır:

$$P(w_1, w_2, w_3, w_4) = P(w_1)\, P(w_2)\, P(w_3)\, P(w_4)$$

$$P(w_4 \mid w_1, w_2, w_3) = P(w_4), \qquad P(\text{geldi} \mid \text{ali okuldan buraya}) = P(\text{geldi})$$

$k = 1$ için her olasılık yalnız bir öncekine bağlıdır:

$$P(w_1, w_2, w_3, w_4) = P(w_2 \mid w_1)\, P(w_3 \mid w_2)\, P(w_4 \mid w_3)$$

$$P(w_4 \mid w_1, w_2, w_3) = P(w_4 \mid w_3), \qquad P(\text{geldi} \mid \text{ali okuldan buraya}) = P(\text{geldi} \mid \text{buraya})$$

$k = 2$ için örneğin w_3, $w_1 w_2$'ye; w_4 ise $w_2 w_3$'e bağlıdır. k istenildiği kadar artırılabilir.

> **Ders sonrası not:** Slayttaki $k = 1$ açılımında $P(w_1)$ çarpanı yazılmamıştır; tam açılım $P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_2)\, P(w_4 \mid w_3)$ olmalıdır. Aynı durum ileride bigram örneğinde de vardır: cümle olasılığı, ilk kelimenin olasılığı olmadan koşullu olasılıkların çarpımı olarak hesaplanmıştır.

Genel olarak, varsayım yokken (zincir kuralı)

$$P(w_i \mid w_1, \ldots, w_{i-1})$$

kullanılır; Markov varsayımıyla bu

$$P(w_i \mid w_{i-k}, \ldots, w_{i-1})$$

olur. Yani $i - k$'dan önceki kelimeler hiç hesaba katılmaz, yalnız son k kelime işe girer.

9.2 Avantaj ve dezavantaj

Soru: k'dan öncesini dikkate almamanın iyi tarafı nedir? **Cevap:** Çok uzun diziler kullanılmayacağı için bu dizilerin derlemde bulunma olasılığı yüksek olur; sıfırlardan büyük ölçüde kurtulunur. Ayrıca hesaplamalar kolaylaşır, hızlanır; gereken RAM miktarı ciddi biçimde azalır, hem zamandan hem bellekten tasarruf edilir.

Soru: Peki kötü tarafı nedir? Bir öğrenci, cümleyi parça parça aldığımızı söyledi; hoca bunu doğru buldu ve slayttaki örnekle açıkladı (ekrandan, 01:30:00):

> XXX geçirdiğim süre içinde çok denedim ama bir türlü FFF öğrenemedim.

FFF ne olabilir? Öğrencilerden "Çince" ve "Fuşi yapmayı" cevapları geldi. Ama bu cevaplar XXX'e aşırı derecede bağlıdır. XXX = "Çin'de" ise FFF = Çince ya da Fuşi yapmayı olabilir. XXX = "Fransa'da" ise FFF'ler bambaşka şeyler olur. FFF'yi belirleyen kelime 5-6 kelime geride durur. Daha da uzakta olabilir: bir romanın başında kahramanın Çin'de olduğu anlatılır, araya sayfalarca olay girer ve sonra bu cümle gelir. Yani dilde bir kelime çok uzaktaki bir şeye bağlı olabilir. Buna uzak bağımlılık (long dependency) denir. k küçük seçilirse k penceresine girmeyen uzak bağımlılıklar yakalanamaz.

Soru (bir öğrenci): Aynı cümle içindeyken Markov ile bunu kolayca yapabiliyoruz, değil mi? **Cevap:** Hoca, Markov ile yapılabildiğini söylemediğini düzeltti: bu tamamen k değerine bağlıdır; k ne kadarsa o kadar uzak bağımlılık modele katılır. Metin illa tek bir cümle olmak zorunda değildir; bir paragraf da alınabilir, bir hikâyenin başı verilip devamı tahmin ettirilebilir. Asıl soru, bir kelimenin olasılığının kendinden önceki kaç token'a bağlı sayıldığıdır. Standart istatistiksel dil modeli k'yı sonsuz kabul eder: kelime metnin en başındaki kelimeye bile bağlıdır. Ama bu sayı çok yükseltilince hesaplama maliyeti korkunç biçimde artar ve uygun veri bulma olasılığı çok azalır.

Slaytta bu özetle şöyle geçer: k seçimi bir ikilemdir; küçük k daha az işlem ve daha az seyreklik demektir, ama uzak bağımlılıklar kaybolur.

10. Seyreklik: Ekonomi Haberleri Örneği

Slayttaki örnek kod `dil_modeli.m` dersin sayfasında da bulunuyor (ekrandan, 01:33:00):

- 1000 ekonomi haberinden bir derlem oluşturuldu.
- 5'ten az geçen kelimeler silindikten sonra yaklaşık 8 bin kelimelik bir sözlük ve yaklaşık 8K×8K'lık bir matris elde edildi.
- Matrisin her hücresi, satırdaki kelimeden sonra sütundaki kelimenin derlemde gelip gelmediğini (kaç kez geldiğini) gösterir.
- Çok seyrek: yaklaşık 64M hücreden yalnız yaklaşık 140K'sı 0 değildir.

Slayttaki görselde kelimeler frekansa göre sıralandığı için matrisin sol üst köşesi yoğun görünür; ama o köşe büyütüldüğünde de hücrelerin çoğunun boş olduğu görülür. Bu, dilde her kelimeden sonra her kelimenin gelmediğini, dağılımın çok seyrek olduğunu gösterir. Hesap: $8000 \times 8000 = 64$ milyon hücrenin yaklaşık 140 bini dolu, yani hücrelerin yaklaşık binde 2'si (ara adım eklendi).

Soru: Şu iki kelimeden sonra şu kelimenin gelmesi için ne olur? **Cevap:** Bir öğrenci ağaç yapısı önerdi; hoca çok düz düşünülürse bir küp olacağını söyledi. Bir eksen birinci kelimeleri, biri ikinci kelimeleri, biri üçüncü kelimeleri gösterir; "şu iki kelimeden sonra şu kelime" küpte bir göz olur. Hücre sayısı 64 milyon $\times\ 8000 = 512$ milyar olur ve bunların çok çok azı sıfırdan farklıdır. k artırıldıkça süreç aşırı derecede seyrekleşir.

Bu matrislerin neden önemli olduğu: Bir cümlenin olasılığını bulurken her seferinde derlemde w_1'i, sonra $w_1 w_2$'yi aratmak makul değildir; örneğin 1 GB'lık bir derlemi her adımda yeniden sorgulamanın anlamı yoktur. Bu yüzden derlemdeki bütün kelimelerin bütün kelimelerle olasılıkları önceden çıkarılır. Bu kadar seyrek bir yapıyı düz bir matriste tutmak da makul değildir; hash gibi yapılarda tutulabilir. Matrisin bir satırı, örneğin "Ali" kelimesinden sonra diğer 7999 kelimenin her birinin kaç kez geçtiğini gösterir. Satır toplamına bölünürse satır toplamı 1 olan olasılıklar elde edilir. "ali okuldan buraya"dan sonraki kelime için ise dört boyutlu, daha da uzun bağlamlar için daha da büyük yapılar gerekir ve bunlarda sıfır olmayan değer bulma olasılığı giderek düşer.

> **Hocanın vurgusu:** Markov varsayımı önemli bir varsayımdır; işi pratikte yapılabilir hâlde tutar. k üzerine hiçbir varsayım yapılmazsa gerçekçi bir derlemde doğru bir olasılık hesabı pek mümkün değildir. k bir yerden kesilmek zorundadır; bunun bedeli uzak bağımlılıkları yakalayamamaktır.

Bu sorunun çözümü ileride görülecek sinirsel dil modelleridir. Bunların ilk yapısı olan RNN'ler, bir durum (state) içinde o ana kadar gelen bütün kelimelerden bilgi tutar ve teoride sonsuz uzunluktaki bağlamları temsil edebilir.

10.1 Örnek çıktılar

Aynı derlemden elde edilen sonuçlar (ekrandan, 01:40:00):

- "ekonomi"den sonra en fazla geçen 5 kelime, yani $P(w_i \mid \text{ekonomi})$ değeri en yüksek 5 kelime: bakanı, ve, ligi, bakanlığı, bankası.
- "para"dan sonra: politikası, cezası, ve, birimlerinin.
- "çok"tan sonra: önemli, daha, büyük, sayıda, iyi.

11. Bigram Modeliyle Cümle Olasılığı

$k = 1$ durumu, yani her kelimenin yalnız bir öncekine bağlı olduğu model bigram modelidir. Slayttaki örnek (ekrandan, 01:41:50):

$$P(\text{fuar otomotiv sektörüne önemli bir hareket getirdi}) = P(\text{otomotiv} \mid \text{fuar}) \cdot P(\text{sektörüne} \mid \text{otomotiv}) \cdot P(\text{ö}$$

Her koşullu olasılık frekanslardan bulunur:

$$P(w_i \mid w_{i-1}) = \frac{fr(w_{i-1}, w_i)}{fr(w_{i-1})}, \qquad \text{örneğin } P(\text{otomotiv} \mid \text{fuar}) = \frac{fr(\text{fuar otomotiv})}{fr(\text{fuar})}$$

Slayttaki tablo (ilk sütun önceki kelime w_{i-1}'dir):

Önceki kelime	$fr(w_{i-1})$	$fr(w_{i-1}, w_i)$	$P(w_i \mid w_{i-1})$
fuar	18	1	1/18
otomotiv	78	4	4/78
sektörüne	35	1	1/35
önemli	560	162	162/560
bir	3940	4	4/3940
hareket	42	1	1/42

Yani derlemde "fuar" 18 kez, "otomotiv" 78 kez, "sektörüne" 35 kez, "önemli" 560 kez, "bir" 3940 kez, "hareket" 42 kez geçmiştir. "fuar otomotiv" 1 kez, "otomotiv sektörüne" 4 kez, "sektörüne önemli" 1 kez, "önemli bir" 162 kez, "bir hareket" 4 kez, "hareket getirdi" 1 kez geçmiştir.

Bu olasılıklar çarpılınca korkunç küçük bir sayı çıkar. Bu yüzden log alınır: slayttaki sonuç

$$\log P(\text{fuar otomotiv sektörüne önemli bir hareket getirdi}) = -21.2067$$

Hoca bunu "e üzeri eksi 21 civarında" diye okudu, yani doğal logaritmadır. Matematiksel olarak çarpımın logaritması yerine logaritmaların toplamı da alınabilir:

$$\log \prod_i P(w_i \mid w_{i-1}) = \sum_i \log P(w_i \mid w_{i-1})$$

Adım adım (ara adım eklendi, python ile hesaplandı):

Olasılık	Değer	ln
$P(\text{otomotiv} \mid \text{fuar}) = 1/18$	0,05556	-2,890
$P(\text{sektörüne} \mid \text{otomotiv}) = 4/78$	0,05128	-2,970
$P(\text{önemli} \mid \text{sektörüne}) = 1/35$	0,02857	-3,555
$P(\text{bir} \mid \text{önemli}) = 162/560$	0,28929	-1,240
$P(\text{hareket} \mid \text{bir}) = 4/3940$	0,00102	-6,893
$P(\text{getirdi} \mid \text{hareket}) = 1/42$	0,02381	-3,738
Toplam	çarpım $\approx 5{,}69 \times 10^{-10}$	$\approx -21{,}29$

> **Ders sonrası not:** Tablodaki değerlerle hesaplanan doğal logaritma yaklaşık $-21{,}29$ çıkar; slayttaki -21.2067 değeri buna yakın ama birebir aynı değildir. Fark büyük olasılıkla slayttaki değerin farklı bir çalıştırmadan ya da yuvarlanmış frekanslardan gelmesindendir.

12. Sıfırlarla Baş Etmek: Yumuşatma

Hoca cümledeki "bir" kelimesini "iki" ile değiştirdi: "fuar otomotiv sektörüne önemli iki hareket getirdi". Bu cümlenin log olasılığı $-\infty$ çıkar (bkz. Örnek Sorular). Nedeni, "önemli iki" (ya da "iki hareket") ifadesinin derlemde hiç geçmemesidir: koşullu olasılık 0 olur, $\log 0 = -\infty$. Yalnız ikili (bigram) yapılarda bile bu sonuç çıkıyorsa, üçlü, dörtlü, beşli yapılarda sıfırlarla çok daha sık karşılaşılır. Üstelik bu durumda cümlenin olasılığı hiç hesaplanamaz.

Çok basit bir çözüm vardır: paydaki her frekansa 1 eklenir, paydaya da sözlüğün boyutu eklenir ki dağılımda çok büyük bir bozulma olmasın. Hoca bunu "bir ekleyerek tahmin" olarak anlattı:

$$P(w_i \mid w_{i-1}) = \frac{fr(w_{i-1}, w_i) + 1}{fr(w_{i-1}) + V}$$

Burada V sözlük boyutudur (bu örnekte yaklaşık 8000). Ekleme bütün olasılıklara yapılır. Böylece hiçbir zaman sıfır elde edilmez; hiç görülmemiş bir ikili bile en az $1/(fr(w_{i-1})+V)$ olasılık alır.

> **Ek bilgi (derste anlatılmadı):** Bu yöntem literatürde Laplace yumuşatması ya da add-one smoothing olarak bilinir. Paydaya V eklenmesinin nedeni, payda her olası sonraki kelime için 1 eklendiğinde olasılıkların toplamının yine 1 olmasını sağlamaktır.

Bu yumuşatmayla hesaplanan değerler:

- "fuar otomotiv sektörüne önemli bir hareket getirdi": yaklaşık -44.
- "fuar otomotiv sektörüne önemli iki hareket getirdi": yaklaşık -49.

Hiç görülmemiş ikili olsa bile artık daha gerçekçi bir karşılaştırma yapılabilir.

Soru: -44 ile -49 arasındaki 5 birimlik fark büyük mü, küçük mü? **Cevap:** Log ölçeğinde 5 birim, olasılıklar arasında e^5 katlık bir farktır. Hoca bunu $e \approx 3$ alarak yaklaşık 3^5 diye hesapladı ve "biri diğerinden 270-300 kat daha büyük" dedi. Bu küçük görünen fark gerçekte büyüktür. Log kullanılmasının nedeni de budur: gerçek olasılıklar "korkunç sıfırlı" sayılardır, onları yazmak ve yorumlamak pek mümkün değildir.

> **Ders sonrası not:** Hoca farkı $3^5 = 243$ ve "270-300 kat" diye tahmin etti. Doğal logaritmada doğrusu $e^5 \approx 148$ kattır. Sonuç yine "büyük bir fark"tır.

13. En Olası Dizilişi Bulmak

Slayttaki örnek (ekrandan, 01:47:40): kelimeler "önemli", "hareket", "bir". Bu üç kelimenin olası dizilişlerinin yumuşatılmış log olasılıkları (P_{LS}):

Diziliş	$\log P_{LS}$
önemli bir hareket	-11.8570
önemli hareket bir	-18.1881
bir önemli hareket	-17.4616
bir hareket önemli	-16.8926
hareket önemli bir	-13.0944
hareket bir önemli	-17.4034

Değer sıfıra ne kadar yakınsa o diziliş o kadar olasıdır. En olası diziliş "önemli bir hareket" (-11.857), ikincisi "hareket önemli bir" (-13.094), en az olası "önemli hareket bir"dir (-18.188).

Soru (slayttan): Yumuşatma kullanmasaydık? **Cevap:** Hoca, dizilişlerin çoğunun büyük olasılıkla $-\infty$ çıkacağını, belki biri hesaplanıp diğerlerinin hesaplanamayacağını söyledi. O zaman dizilişlerin birbirine göre iyiliği hakkında hiçbir şey söylenemezdi.

Slaytta bir başka uygulama da verildi: "Şu kelimelerle bir cümle kurun." Hoca bunu bir düşünme sorusu olarak bıraktı (bkz. Örnek Sorular).

Örnek Sorular

Soru 1 (derste sözlü, 00:12:26): Düz metinden "katil, kurban, sebep, suç aleti" gibi bir tablo çıkarabilmek ne işe yarar?

> **Cevap (derste verildi):** Veri çok daha işlenebilir hâle gelir; veritabanı kurulabilir (örneğin suç aletleri ya da katiller veritabanı) ve istatistik çıkarmak kolaylaşır. Bir milyon paragraftan böyle tablolar çıkarılırsa metinler okunmadan hızlı ve sağlam istatistik elde edilir.

Soru 2 (ekrandan, 00:22:10; derste sözlü, 00:22:26): "X'in yeri Y'dir" ilişkisi için "x y nin sınırları içerisindedir" ve "x/y" şablonları veriliyor. Hangisi daha fazla bilgi döndürür? Hangisinin döndürdüğü bilgi daha doğrudur?

> **Cevap (derste verildi):** "x/y" çok daha fazla sonuç döndürür, çünkü eğik çizginin geçtiği her yerde iki yanındaki kelimeler alınır; üretkenliği yüksektir. "x y nin sınırları içerisindedir" çok daha az sonuç döndürür ama döndürdüklerinin doğru olma olasılığı çok daha yüksektir; güvenilirliği yüksektir. Üretkenlik ve güvenilirlik birbiriyle çelişir.

Soru 3 (derste sözlü, 00:36:23): Stack Overflow ya da GitHub içeriğini özümseyip soruları kaynak göstermeden doğrudan cevaplayan bir sistem olsa, insanlar bu sitelere içerik koymaya devam eder mi?

> **Cevap:** Açık uçlu bir tartışma sorusuydu; tek bir doğru cevap verilmedi. Görüşler Bölüm 5.2'de özetlendi. Hoca, kaynağın gösterilmemesinin ve bunun zorluğunun (açıklanabilir yapay zekâ) asıl sorun olduğunu vurguladı.

Soru 4 (ekrandan, 01:20:00; derste sözlü, 01:17:33): $P(\text{geldi} \mid \text{ali okuldan buraya}) = \dfrac{\text{frekans(ali okuldan buraya geldi)}}{\text{frekans(ali okuldan buraya)}}$ değeri hangi aralıktadır? 1 olması için ne gerekir?

> **Cevap (derste verildi):** Değer 0 ile 1 arasındadır; negatif olamaz, 1'den büyük de olamaz, çünkü pay paydadan büyük olamaz. "ali okuldan buraya geldi" hiç geçmiyorsa 0'dır. 1 olması için iki frekansın eşit olması, yani "ali okuldan buraya"dan sonra her seferinde "geldi" gelmesi gerekir.

Soru 5 (ekrandan, 01:30:00): "XXX geçirdiğim süre içinde çok denedim ama bir türlü FFF öğrenemedim." cümlesinde FFF ne olabilir? Bu örnek Markov varsayımı hakkında neyi gösterir?

> **Cevap (derste verildi):** XXX = "Çin'de" ise FFF = "Çince" ya da "Fuşi yapmayı" olabilir; XXX = "Fransa'da" ise FFF başka şeyler olur. FFF'yi belirleyen kelime 5-6 kelime geride, hatta bir romanda sayfalarca geride olabilir (uzak bağımlılık). k küçük seçilirse bu bağımlılık yakalanamaz.

Soru 6 (ekrandan, 01:33:00; derste sözlü, 01:33:47): 8000 kelimelik sözlükle "bir kelimeden sonra gelen kelime" için 8K×8K'lık bir matris gerekiyor. "İki kelimeden sonra gelen kelime" için ne gerekir ve kaç hücresi olur?

> **Cevap (derste verildi):** Üç boyutlu bir küp gerekir: bir eksen birinci, biri ikinci, biri üçüncü kelimeleri gösterir. Hücre sayısı $8000^3 = 64$ milyon $\times 8000 = 512$ milyar olur ve bunların çok azı sıfırdan farklıdır.

Soru 7 (derste sözlü, 01:43:36): "fuar otomotiv sektörüne önemli iki hareket getirdi" cümlesinin (yumuşatmasız) bigram log olasılığı neden $-\infty$ çıkar?

> **Cevap (derste verildi):** "önemli iki" (ya da "iki hareket") ifadesi derlemde hiç geçmediği için ilgili koşullu olasılık 0 olur; çarpım 0, logaritması $-\infty$ olur.

Soru 8 (derste sözlü, 01:46:04): Yumuşatmayla iki cümlenin log olasılığı -44 ve -49 bulundu. Aradaki 5 birimlik fark olasılıklar arasında kaç katlık bir farktır?

> **Cevap (derste verildi, düzeltilerek):** Doğal logaritmada fark e^5 kattır. Hoca bunu yaklaşık 3^5, "270-300 kat" diye tahmin etti; doğrusu $e^5 \approx 148$ kattır (python ile hesaplandı). Küçük görünen log farkı gerçekte büyük bir farktır.

Soru 9 (ekrandan, 01:47:40): "önemli, hareket, bir" kelimelerinin dizilişlerinin log olasılıkları verildiğine göre en olası diziliş hangisidir? Yumuşatma kullanılmasaydı ne olurdu?

> **Cevap (derste verildi):** En olası diziliş, log olasılığı sıfıra en yakın olan "önemli bir hareket"tir (-11.857). Yumuşatma kullanılmasaydı dizilişlerin çoğu $-\infty$ çıkacak ve aralarında karşılaştırma yapılamayacaktı.

Soru 10 (ekrandan, 01:47:40; derste sözlü, 01:48:39): Size 20 kelimelik bir kelime torbası veriliyor (aslında 20 kelimelik bir cümle karıştırılmış). İstediğiniz k değerine sahip bir istatistiksel dil modeliniz var. Bu kelimelerin hepsini kullanarak anlamlı bir cümleyi makineye nasıl kurdurursunuz? Bunu yaparken ne gibi sorunlarla karşılaşırsınız?

> **Çözüm (derste verilmedi):** Hoca bu soruyu sonraki dersin başında tartışılmak üzere bıraktı. En doğrudan yol, bütün sıralamaları dil modeliyle puanlayıp en yüksek log olasılıklı olanı seçmektir (Bölüm 13'teki üç kelimelik örneğin genellemesi). Ama 20 kelimenin $20! \approx 2{,}43 \times 10^{18}$ farklı sıralaması vardır (python ile hesaplandı); hepsini puanlamak pratikte mümkün değildir. Bu yüzden aramayı daraltan bir yöntem (örneğin her adımda en olası birkaç kısmi diziyi tutarak ilerlemek) gerekir; ayrıca küçük k ile uzak bağımlılıklar ve seyreklik yine sorun olur. Bu son kısım ajanın önerisidir, derste anlatılmadı.

Dersin Toparlaması

Bu derste önce hayat bilgisi veritabanları konusu, bilgi çıkarımıyla tamamlandı. Bilgi çıkarımı, düz metni tablo gibi yapısal bir biçime çevirmektir. Regex, yazım dilindeki şablonlar, öğelerine ayrılmış metinler ve HTML kalıpları bu işin başlıca yöntemleridir. Bütün şablon tabanlı yöntemlerde üretkenlik ile güvenilirlik birbiriyle çelişir. Şablonlar doğru olduğu bilinen tohum ikililerden otomatik bulunabilir; ürettikleri ikililer de insan etiketi ve makine öğrenmesiyle yaklaşık %85 başarıyla süzülebilir. Web sayfalarından bilgi çıkarımı konusu, üretici modellerin gönüllü içerik üzerindeki etkisi ve kaynak gösterme (açıklanabilir yapay zekâ) sorunu üzerine bir sınıf tartışmasına açıldı.

Dersin ikinci yarısında "Anlamsal Uzaylar" bölümüne istatistiksel dil modelleriyle girildi. Bu modeller bir cümlenin ve sonraki kelimenin olasılığını zincir kuralı ve derlem frekanslarıyla hesaplar. Dil çok seyrek olduğu için çoğu frekans sıfırdır; Markov varsayımı bağlamı son k kelimeyle sınırlayarak hesabı pratik hâle getirir, ama uzak bağımlılıkları kaybettirir. Bigram örneğinde cümle olasılığı log toplamıyla hesaplandı, hiç görülmemiş ikililerin yol açtığı $-\infty$ sorunu bir ekleyerek yumuşatmayla çözüldü ve aynı model kelimelerin en olası dizilişini bulmak için kullanıldı. Hoca, uzak bağımlılık sorununun çözümünün ileride görülecek sinirsel dil modelleri (RNN'ler) olduğunu söyledi ve 20 kelimelik kelime torbasından cümle kurma sorusunu sonraki derste tartışılmak üzere bıraktı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.